Práctica 4: pruebas unitarias

Ticket de estacionamiento

Nombre: Román Ismael Contreras Morales

1 Reglas del modelo

Las reglas que definen cómo debe operar el sistema son:

R1. Los minutos deben ser un número entero mayor o igual que 0.
R2. Los tipos de cliente válidos son `"normal"` y `"frecuente"`.
R3. Si el boleto está perdido, el costo final es de $300.00, sin importar los minutos ni el tipo de cliente.
R4. Si los minutos son 0, el costo es $0.00.
R5. De 1 a 15 minutos, el costo es $0.00.
R6. De 16 a 60 minutos, el costo es $20.00.
R7. Después de 60 minutos se cobran $20.00 por la primera hora y $15.00 por cada hora adicional iniciada.
R8. Un cliente `"frecuente"` recibe 10 % de descuento sobre el costo calculado, excepto cuando perdió el boleto.
R9. El costo nunca puede ser negativo.
R10. Si los minutos son negativos, el sistema debe lanzar un error.
R11. Si el tipo de cliente no es válido, el sistema debe lanzar un error.

2 Tabla de casos

A continuación detallamos la tabla de casos que propusimos para las pruebas.

Los valores esperados se calculan directamente a partir de las reglas.

La columna de entrada se lee como `tipo, minutos` y en caso de que el boleto se haya perdido, se agrega `perdido`, de lo contrario se omite.

Caso	Entrada	Esperado	Regla	Justificación
1	normal, 12 min	$0.00	R5	Caso normal dentro de la franja sin cargo.
2	normal, 70 min	$35.00	R7	Una hora adicional iniciada.
3	normal, 500 min, perdido	$300.00	R3	Tarifa fija sin importar los minutos.
4	normal, 0 min	$0.00	R4	Límite inferior del dominio.
5	normal, 14 min	$0.00	R5	Último minuto antes del cambio de franja.
6	normal, 15 min	$0.00	R5	Límite superior de R5.
7	normal, 16 min	$20.00	R6	Primer minuto de R6.
8	normal, 17 min	$20.00	R6	Confirma que se sigue en R6.
9	normal, 60 min	$20.00	R6	Límite superior de R6.
10	normal, 61 min	$35.00	R7	Primera hora adicional iniciada.
11	normal, 120 min	$35.00	R7	Hora adicional completa, sin cargo extra.
12	normal, 121 min	$50.00	R7	Segunda hora adicional iniciada.
13	normal, −1 min	error	R10	Valor negativo en el límite.
14	normal, −100 min	error	R10	Negativo alejado del límite.
15	normal, 15.5 min	error	R1	Minutos no enteros. Sólo aplica en Python.
16	`""`, 0 min	error	R11	Sin tipo de cliente.
17	`"vip"`, 0 min	error	R11	Tipo de cliente diferente a los establecidos.
18	`"Normal"`, 0 min	error	R11	Distingue mayúsculas.
19	frecuente, 50 min	$18.00	R6 + R8	Descuento sobre la primera hora.
20	frecuente, 121 min	$45.00	R7 + R8	Descuento sobre horas adicionales.
21	frecuente, 500 min, perdido	$300.00	R3 + R8	El descuento no aplica al boleto perdido.
22	frecuente, 61 min	$31.50	R7 + R8	Resultado con centavos.
23	varias entradas válidas	>= $0.00	R9	El costo nunca es negativo.
24	normal, 180 min	$50.00	R7	Caso adicional: tercera hora completa.
25	normal, 181 min	$65.00	R7	Caso adicional: tercera hora iniciada.
26	frecuente, 180 min	$45.00	R7 + R8	Caso adicional: descuento sobre hora completa.
27	frecuente, 181 min	$58.50	R7 + R8	Caso adicional: descuento con centavos.

3 Reglas cubiertas

Con los casos propuestos se cubren todas las reglas. En la siguiente tabla mostramos qué casos cubren cada regla.

Regla	Contenido	Casos
R1	Minutos enteros	15
R2	Tipos válidos: normal y frecuente	1–12, 19–22
R3	Boleto perdido cuesta $300.00	3, 21
R4	0 minutos cuestan $0.00	4
R5	1 a 15 minutos cuestan $0.00	1, 5, 6
R6	16 a 60 minutos cuestan $20.00	7, 8, 9
R7	Horas adicionales iniciadas	2, 10, 11, 12, 24–27
R8	Descuento del 10 % al cliente frecuente	19, 20, 22, 26, 27
R9	El costo nunca es negativo	23
R10	Minutos negativos lanzan error	13, 14
R11	Tipo de cliente inválido lanza error	16, 17, 18

4 Evidencia

4.1 Python

En Python las pruebas se implementaron con `pytest`. Para evitar problemas con el manejo de entornos virtuales utilizamos `uv`, por lo que para correr las pruebas usamos el comando: `cd python && uv run pytest -v --tb=line`

Este comando produce la salida detallada con el resultado de todas las pruebas. A continuación mostramos un extracto de la salida, mostrando principalmente los casos donde se presenta una falla.

```
$ cd python && uv run pytest -v --tb=line
...
=================================================== short test summary info ====================================================
FAILED test_estacionamiento.py::test_costo_cero_hasta_quince_minutos[15min] - assert 20.0 == 0.0 ± 1.0e-12
FAILED test_estacionamiento.py::test_horas_adicionales_iniciadas[61min-una-hora-iniciada] - assert 20.0 == 35.0 ± 3.5e-05
FAILED test_estacionamiento.py::test_horas_adicionales_iniciadas[70min-una-hora-iniciada] - assert 20.0 == 35.0 ± 3.5e-05
FAILED test_estacionamiento.py::test_horas_adicionales_iniciadas[121min-dos-horas-iniciadas] - assert 35.0 == 50.0 ± 5.0e-05
FAILED test_estacionamiento.py::test_cliente_frecuente_paga_descuento_sobre_horas_adicionales - assert 31.5 == 45.0 ± 4.5e-05
FAILED test_estacionamiento.py::test_cliente_frecuente_genera_costo_decimal - assert 18.0 == 31.5 ± 3.1e-05
FAILED test_estacionamiento.py::test_boleto_perdido_no_lleva_descuento - assert 270.0 == 300.0 ± 3.0e-04
FAILED test_estacionamiento.py::test_minutos_no_enteros_lanzan_error - Failed: DID NOT RAISE ValueError
FAILED test_estacionamiento.py::test_tercera_hora_adicional[181min-normal-hora-iniciada] - assert 50.0 == 65.0 ± 6.5e-05
FAILED test_estacionamiento.py::test_tercera_hora_adicional[181min-frecuente-hora-iniciada] - assert 45.0 == 58.5 ± 5.8e-05
================================================= 10 failed, 19 passed in 0.05s ================================================
```

4.2 Java

Para el caso de Java utilizamos Maven para ejecutar las pruebas. De nueva cuenta, mostramos un extracto de la salida, enfocándonos en los casos con fallas.

```
$ cd java && mvn test
[INFO] Results:
[INFO]
[ERROR] Failures:
[ERROR]   EstacionamientoTest.boletoPerdidoNoLlevaDescuento:163 expected: <300.0> but was: <270.0>
[ERROR]   EstacionamientoTest.clienteFrecuenteCostoDecimal:148 expected: <31.5> but was: <18.0>
[ERROR]   EstacionamientoTest.clienteFrecuenteDescuentoHorasAdicionales:133 expected: <45.0> but was: <31.5>
[ERROR]   EstacionamientoTest.costoCeroHastaQuinceMinutos:40 expected: <0.0> but was: <20.0>
[ERROR]   EstacionamientoTest.horasAdicionalesIniciadas:82 expected: <35.0> but was: <20.0>
[ERROR]   EstacionamientoTest.horasAdicionalesIniciadas:82 expected: <35.0> but was: <20.0>
[ERROR]   EstacionamientoTest.horasAdicionalesIniciadas:82 expected: <50.0> but was: <35.0>
[ERROR]   EstacionamientoTest.terceraHoraAdicional:242 expected: <65.0> but was: <50.0>
[ERROR]   EstacionamientoTest.terceraHoraAdicional:242 expected: <58.5> but was: <45.0>
[INFO]
[ERROR] Tests run: 28, Failures: 9, Errors: 0, Skipped: 0
```

5 Hallazgos

Las pruebas que fallan se concentran en cuatro diferencias entre el modelo y la implementación entregada:

- R5: una estancia de 15 minutos se cobra como $20.00, cuando el modelo indica que hasta 15 minutos el costo es $0.00.
- R7: las estancias posteriores a la primera hora que no completan una hora exacta pagan menos de lo que corresponde. Se observa a los 61, 70, 121 y 181 minutos, y en el caso de cliente frecuente el monto con descuento también queda por debajo de lo esperado.
- R3 y R8: cuando el boleto está perdido y el cliente es frecuente, se cobra $270.00 en lugar de $300.00. El modelo fija la tarifa de boleto perdido en $300.00 y excluye el descuento en ese caso.
- R1: en Python se aceptan minutos no enteros, como 15.5, sin lanzar el error que exige el modelo. En Java no puede probarse porque el parámetro es `int`.

6 Análisis

1. **¿Qué diferencia hay entre un caso normal y un caso frontera?**

   Un caso normal pretende probar un uso típico del programa, con valores de entrada frecuentes o comunes. En cambio, un caso frontera se diseña *ex profeso* para verificar que el resultado cambie exactamente en el punto donde debe cambiar. Para esto se pueden tomar valores de entrada muy cercanos al punto donde alguna regla fuerza un cambio de comportamiento.

   En nuestro caso, por ejemplo 15 y 16 minutos, o 60 y 61.
2. **¿Cuál de sus pruebas considera más importante y por qué?**

   La prueba de 61 minutos con cliente normal. Es la frontera entre R6 y R7 y distingue una hora completa de una hora iniciada, que es justo donde falla la implementación. Su variante con cliente frecuente, a 61 minutos, conecta además R7 con R8 y permite ver si el descuento se aplica sobre el costo correcto.
3. **¿Encontró algún comportamiento de la implementación que no coincida con el modelo?**

   Sí, los detallamos en la sección 5.
4. **¿Una suite con 100 % de pruebas aprobadas demuestra que el programa es correcto? Explique.**

   No. Una suite de pruebas no puede garantizar la corrección, pues unicamente revisa una muestra finita, no todas las entradas posibles, y además, si los casos se derivan del propio código podría convalidar sus errores. Si lo que se busca son garantías más fuertes, sería necesario dar el giro hacia la verificación formal, que demuestra propiedades para todas las entradas a partir de un modelo matemático, siempre que ese modelo represente correctamente las reglas del problema.
5. **Si una IA generara automáticamente 50 pruebas, ¿qué tendría que revisar una persona antes de confiar en ellas?**

   Primero, que cada valor esperado provenga del modelo y no de ejecutar el código. Después, que cubran fronteras e inválidos, no sólo los casos principales. También hay que revisar que las pruebas no sean tautológicas ni duplicadas, que las excepciones esperadas sean las correctas, que los nombres indiquen qué regla prueban y que fallen cuando deben fallar.